

【机试C++】入门语法、STL、一些注意事项

参考文章

- [从C语言到C++竞赛速成指南（蓝桥杯专用）](#)
- [【第十四届蓝桥杯考前速成】必考知识点及代码模板总结，看完至少多拿50分](#)

基础知识

数据类型

```
int           // 整型（4字节，常用整数类型）
short         // 短整型（2字节，范围较小）
long          // 长整型（4或8字节，取决于系统）
long long     // 长长整型（8字节，表示更大范围整数），防止溢出

char          // 字符类型（1字节，存字符或ASCII码）

float         // 单精度浮点数（4字节，精度较低）
double        // 双精度浮点数（8字节，精度较高，默认小数类型）

void          // 空类型（无返回值 / 无类型指针 void*）

bool          // 布尔类型（true / false，占1字节）

auto x = 10;   // 自动类型推导（C++11）
```

C

数据类型转换

static_cast

```
static_cast<目标类型>(表达式)

//示例
double a = 3.14;
int b = static_cast<int>(a); // 3
```

C

小括号直接强转

```
(int)a

//示例
double x = 5.9;
int y = (int)x; // 5
```

C

char与ASCII

```
// char → ASCII
char c = 'A';
cout << (int)c; // 65

// ASCII → char
int x = 65;
cout << (char)x; // A
```

C

char → int

```
char c = '7';
int x = c - '0'; // 7
```

C

int → char

```
// 按照ASCII
int x = 65;
char c = x; // A
char c = (char)x; // 强转 → A
// 5 → '5'
char c = x + '0';
```

基本类型 → string

```
int num = 123;
string s = to_string(num);
```

string → int

```
string str = "123";
int num = stoi(str);
```

字符串逐位拆数字 string → int[]

```
vector<int> a;
string s;
cin >> s;
for(char c : s){
    a.push_back(c - '0');
}
// 或者
for (int i = 0; i < s.size(); i++) {
    int a = s[i] - '0';
    list.push_back(a);
}
```

String 与 char[]

```
// String → char[]
string s = "456987412dfwerfd";
vector<char> array(s.begin(), s.end());
// 或者
char array[100];
strcpy(array, s.c_str()); // #include <cstring> 需要使用这个头文件

// char[] → String
char array[] = "abcde";
string str(array);
```

变量与常量

```
//变量
int a = 10;
//常量: 不可修改
const int b = 20;
```

对比 constexpr 和 const

- const : 只读变量, 不保证一定是编译期常量
- constexpr : 必须在编译期计算出结果

```
int x = 100;
const int a = x + 1; // 运行时才能确定

constexpr int MOD = 1000000007; // 必须在编译期计算出结果
```

define

define 是 预处理指令 (preprocessor directive) , 在编译之前起作用。本质是文本替换工具, 不是变量, 也不是函数。

把 long long 简写成 ll

```
#define ll long long

// 示例
ll a = 100000;
ll b = 100000;
cout << a * b;    // 正确 (1e10)
```

C

常见循环宏

```
#define rep(i,a,b) for(int i=a;i≤b;i++)
#define per(i,a,b) for(int i=a;i≥b;i--)

// 示例
rep(i, 1, 5) {
    cout << i << " ";
}
```

全局常量写法

```
#define INF 0x3f3f3f3f
#define MOD 1000000007

// 示例
int dist[1000];
rep(i, 0, 999) dist[i] = INF; // 数组初始化
```

C++

常用头文件

```
#include <bits/stdc++.h> // 万能头文件 (竞赛推荐)
using namespace std;    // 使用标准命名空间

// 或使用传统头文件
#include <iostream>      // 输入输出
#include <vector>         // 动态数组
#include <algorithm>     // 算法库
#include <cstring>       // 字符串, C++对C语言字符串函数的封装版本
#include <string>         // 字符串
#include <cctype>         // 字符处理
#include <cmath>          // 数学运算
```

C++

运算符

逻辑运算符 (条件判断)

```
&& // 与 (and)
||  // 或 (or)
!   // 非 (not)
```

C

位运算符

```
& // 按位与
| // 按位或
^ // 按位异或
~ // 按位取反
<< // 左移 (×2)
>> // 右移 (÷2)
```

注意：求一组数据的异或和的时候，需要起始值为0，即求 $x_1 \oplus x_2 \oplus x_3$ 时，令 `sum = 0`，可以等于求sum与这几个变量的连续异或的结果

条件运算符

```
// 条件 ? 表达式1 : 表达式2
int max = (a > b) ? a : b;
```

数组

比vector更快，长度固定，不可以动态扩展

```
//创建数组
int a[1000];
int a[100][100];
int a[100][];
int a[3] = {0,1,2};
int a[] = {1,2,3};

//初始化
int a[1000] = {0}; // 全部初始化为0

#include <cstring>
memset(a, 0, sizeof(a)); // 全0
memset(a, -1, sizeof(a)); // 全-1
memset(a, 0x3f, sizeof(a)); // INF (常用)
memset(a, 13, sizeof(a)); // int = 4字节, 把 a 所占的所有字节, 每个字节都设置成 13, 即 00001101 00001101 00001101 00001101
```

`memset` 是 `cstring` 头文件中的函数，c语言中延续来的，`string` 头文件中没有

数组大小必须是编译期常量，标准C++不允许用变量定义数组长度

```
int n;
cin >> n;
int a[n]; // ✗ 标准C++不支持
```

vector允许用变量定义数组长度，是动态数组。

```
int n;
cin >> n;
vector<int> a(n);
```

写法	用变量定义数组长度	说明
<code>int a[n]</code>	✗ C++标准不支持	依赖编译器
<code>new int[n]</code>	✓	需要手动释放
<code>vector a(n)</code>	✓	最推荐

sizeof()

表示整个数组占用的字节数，不是元素个数

```
int a[3] = {0,1,2};
sizeof(a) = 3 * sizeof(int) = 12 (通常)
```

C++里没有像 Java 那样统一的 length() 函数来获取数组长度，直接写出数组长度进行遍历

```
int a[3] = {0,1,2};
int b[] = {1,2,3};
for(int i = 0; i < 3; i++){
    cout<<a[i]<<" ";
}
```

指针

指针基础内容

```
int a = 10;
int* p = &a;    // p存放a的地址

&a    // 取地址
*p     // 取值（解引用）

cout << p;    // 地址
cout << *p;   // 10

*p = 20;    // 通过指针修改变量a

int* p = nullptr; // 表示不指向任何地址（C++11推荐）
```

指针与数组

```
int a[3] = {1,2,3};
int* p = a; // *p 和 a 指向同一块内存，改*p就是改a
// 本质
a == &a[0]

cout << *(p + 0); // a[0]
cout << *(p + 1); // a[1]

// 指针自增
p++;    // 指向下一个元素
p--;

// 使用指针遍历数组
for(int* i = a; i < a + 3; i++) {
    cout << *i << " ";
}
```

函数与指针

```
// 定义
void func(int* p) {
    *p = 100;
}

//调用
int a = 10;
func(&a);

// 数组作为参数，传参的本质
void func(int a[])
void func(int* a) // 二者等价
```

结构体与指针（链表、树）

```
// 定义
struct Node {
    int val;
    Node* next;
};
// 访问
Node* p;
p->val;
```

C

引用

```
int a = 10;
int &p = a; // p 是 a 的引用 (别名), p 和 a 是同一个变量
```

C

p 和 a 完全一样, a 和 p 指向同一块内存, 改p就是改a

常用方法

快速输入输出

```
#include <iostream>

int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);
}
```

C

代码	作用
<code>ios::sync_with_stdio(0)</code>	关闭 C++ 与 C 输入输出同步, 即 <code>cin/cout</code> 不再和 <code>scanf/printf</code> 同步, 提高 <code>cin/cout</code> 速度
<code>cin.tie(0)</code>	解除 <code>cin</code> 与 <code>cout</code> 绑定, 减少刷新 (cin输入内容时自动刷新输出缓冲区)
<code>cout.tie(0)</code>	解除 <code>cout</code> 和其他流的绑定。 , 通常不用写

规范输出

头文件

```
#include <iomanip>
```

C

多个数字格式化: double保留3位小数

```
cout << fixed << setprecision(3)
    << a << " " << b << endl; // 小数部分保留3位, 1.235

cout << setprecision(3) << x; // 总共保留3位有效数字, 1.23
```

C

设置字宽, 前面空格补齐0

```
cout << setw(5) << x; // 输出宽度5, 默认右对齐
cout << left << setw(6) << 123; // 左对齐
cout << setfill('0') << setw(2) << x; // 前导0
```

C

其他函数

```
#include <iomanip>
```

```
fixed           // 小数形式，配合setprecision(n)使用
setprecision(n) // 保留n位小数
setw(n)         // 设置输出宽度，只影响紧邻setw()函数的下一个数，如果连续输出两个，第二个不受控制
setfill('c')    // 设置填充字符
left            // 左对齐
right          // 右对齐
scientific      // 科学计数法，如 cout << scientific << x;
boolalpha      // bool输出true/false
```

输出时间格式： 03:05

```
int h=3,m=5;

cout<<setfill('0')
    <<setw(2)<<h
    <<":"
    <<setw(2)<<m;
```

char

```
#include<cctype> // 头文件

toupper(c); // 转成大写
tolower(c); // 转成小写

isupper(c); // 是否是大写字母
islower(c); // 是否是小写字母
isalpha(c); // 是否是字母
isdigit(c); // 是否是数字
```

struct 结构体

```
// 定义结构体
struct score{
    int yuwen, shuxue, total;
};

// 声明结构体变量
score s;

// 访问结构体成员
s.yuwen = 90;
s.shuxue = 95;
s.total = s.yuwen + s.shuxue;

// 在定义结构体的同时声明变量
struct score{
    int yuwen, shuxue, total;
} s;

// 创建并初始化一个结构体的两种方法
score s = {90, 95, 185};
score s{90, 95, 185};
```

结构体数组

```
// 创建一个结构体数组
score a[5];

// 创建一个结构体数组并初始化
score a[5] = {
    {90, 95, 185},
    {85, 90, 175},
    {88, 92, 180},
    {95, 96, 191},
    {80, 85, 165}
};
```

如果使用vector存储结构体

```
vector<score> a = {
    {90, 95, 185},
    {85, 90, 175},
    {88, 92, 180}
};

vector<score> a;
a.push_back({90, 95, 185});
a.push_back({85, 90, 175});
```

STL

STL (Standard Template Library) , 即标准模板库, 是一个具有工业强度的, 高效的C++程序库。它被容纳于C++标准程序库 (C++ Standard Library) 中, 是ANSI/ISO C++标准中最新的也是极具革命性的一部分。该库包含了诸多在计算机科学领域里所常用的基本数据结构和基本算法。

string 字符串

```
#include <string>    //头文件
using namespace std;

string s1 = "Helloworld";
string s2 = "H23456789";
cout << (s1 < s2) << endl; // 支持比较运算符, 按照ASCII码值比较, 是为1, 否为0, 比较首个不同的字符

string s '*'
;    // 输出: **, 重复某个字符n遍, 不可以应用到字符串上, 比如string s(3, "ab"), 这是不可以的

cout << s1.length() << endl;    // 长度
cout << s2.size() << endl;      // 长度

cout << s1.empty() << endl;    // 判断是否为空, 是为1, 否为0
s1.clear();                    // 清空 (输出的时候啥也没有, 如果后面没有其他内容, 本行为空行; 如果有其他内容, 本行只
显示后面的内容, 不换行)

s1.append("123456");           // 追加, 字符串
s1.push_back('c');             // 追加, 字符
s1 += "ssss";                  // 拼接

string s3 = s1.substr(1,3);     // 截取子串, 两个参数分别是: 开始截取的位置, 截取的长度
string s4 = s1.substr(1,15);    // 截取子串, 超出长度, 输出索引为1到最后的所有字符串
string s5 = s1.substr(1);       // 截取子串, 从下标为1开始输出

int idx = s1.find("l");         // 查找子串首次出现的索引
s1.replace(1, 4, "34");         // 替换, 参数: 开始替换的位置, 替换掉的子串的长度, 新子串
s1.insert(3, "222");           // 插入, Helloworld -> Hel222loworld

s1.erase(2, 6);                // 删除, 两个参数分别是: 开始删除的位置, 长度
s1.erase(2, 15);              // 删除, 超出长度, 从起始位置到最后全部删除

// 以下内容不需要加string头文件
int num = stoi("1234");        // string ->int, C++11 新增函数
string str = to_string(num);   // int ->string

char c[] = "1234";
int num2 = atoi(c);            // char[] / string ->int, C 语言中的字符串转整数函数
int num3 = atoi("12345");
```

按字符将字符串分割成多个字符串

```
#include <iostream>
#include <sstream>
#include <vector>
using namespace std;

string s = "18-8-33";
vector<string> ans;
stringstream ss(s);
string x;

while(getline(ss, x, '-')) ans.push_back(x); // 分割并存入vector
for(string x : ans) cout << x << endl; // 输出
```

补充: `cstring`, `memset` 是 `cstring` 头文件中的函数, c语言中延续来的, `string` 头文件中没有

```
#include <cstring>
//初始化数组
int a[1000] = {0}; // 全部初始化为0
memset(a, 0, sizeof(a)); // 全0
memset(a, -1, sizeof(a)); // 全-1
memset(a, 0x3f, sizeof(a)); // INF (常用)
memset(a, 13, sizeof(a)); // int = 4字节, 把 a 所占的所有字节, 每个字节都设置成 13, 即 00001101 00001101
00001101 00001101
```

algorithm 算法

常用函数

```
#include <algorithm>

sort(begin, end) // 对区间进行排序, 默认升序
sort(a, a + n); // 对数组a的[0, n)进行升序排序
sort(a + 1, a + 1 + n); // 对数组a的[1, n]进行升序排序
sort(a.begin() + 1, a.end()); // 对 vector<int> a 的[1, n]进行升序排序
sort(begin, end, greater<int>()) // 降序排序
sort(a + 1, a + 1 + n, greater<int>()); // 对数组a的[1, n]进行倒序排序
reverse(begin, end) // 反转区间元素
swap(a, b) // 交换两个元素

max(a, b) // 返回两个数中的最大值
min(a, b) // 返回两个数中的最小值
max({a,b,c}) // 返回多个数中的最大值
min({a,b,c}) // 返回多个数中的最小值

find(begin, end, x) // 查找元素x, 返回迭代器
count(begin, end, x) // 统计元素x出现次数
binary_search(begin, end, x) // 二分查找, 存在返回true, 否则false

lower_bound(begin, end, x) // 返回第一个 ≥ x 的位置
upper_bound(begin, end, x) // 返回第一个 > x 的位置
nth_element(begin, mid, end) // 将第mid位置元素放到正确位置, 左边 ≤ 它, 右边 ≥ 它

unique(begin, end) // 去除相邻重复元素, 返回新末尾位置

next_permutation(begin, end) // 生成下一个字典序排列
prev_permutation(begin, end) // 生成上一个字典序排列

fill(begin, end, x) // 将区间所有元素赋值为x
copy(begin1, end1, begin2) // 复制区间元素

rotate(begin, mid, end) // 旋转区间, 将mid作为新的开头

partition(begin, end, func) // 按条件划分区间

accumulate(begin, end, init) // 求区间和 (需要#include<numeric>)
```

lower_bound 函数

默认用于 升序数组, 返回第一个 $\geq x$ 的位置。

- 下面代码中查找的范围是 $[a, a+n)$ 或者 $[a + 1, a + 1 + n)$
- 找不到大于等于 x 时, 返回的位置是 $a + n$, 即数组尾部的后一位
- `lower_bound` 返回的是指针 / 迭代器, 需要 `-a` 转换一下, 即计算与 首地址 之间相差多少个元素 (与首地址的相对距离), 才得到元素位置

```
int pos = lower_bound(a, a + n, x) - a; // 待查数x
int pos = lower_bound(a + 1, a + 1 + n, x) - a;
```

unique 去重函数

unique() 只能去除 相邻重复元素，不能直接去除所有重复元素。

- 有序数组/vector → 直接使用 unique() 去重
- 无序数组/vector → 需要先排序，再使用 unique()

vector去重

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main(){
    vector<int> a = {1, 2, 2, 3, 3, 3, 4};
    a.erase(unique(a.begin(), a.end()), a.end()); // unique 返回的是新的结尾位置
    for(int x:a) cout << x << " ";
    return 0;
}
```

unique：返回的是新的结尾位置，但是 vector 的 size 没变。
a.erase(it, a.end())：删除后面的无效元素。

普通数组去重

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main(){
    int a[] = {1,2,2,3,3,4};
    int n = 6; // int n = sizeof(a)/sizeof(a[0]);
    sort(a, a + n);
    int len = unique(a, a + n) - a; // 获取有效数据长度。数组如果是从1开始存的，则为 unique(a + 1, a + n + 1) - (a + 1);
    for(int i = 0; i < len; i++) cout << a[i] << " ";
}
```

vector 动态数组

C

```
#include <vector>    // vector头文件

vector<int> a;        // 创建空vector
vector<int> a(n);     // 创建大小为n的vector, 元素默认初始化为0
vector<int> a(n, x);  // 创建大小为n的vector, 所有元素初始化为x
vector<int> a = {1,2,3}; // 列表初始化
vector<int> b(a);     // 用已有vector初始化
vector<int> b(a.begin(), a.end()); // 使用迭代器范围初始化
vector<vector<int>> a(n, vector<int>(m)) // 创建n行m列二维vector, 元素初始化为0
vector<vector<int>> a(n, vector<int>(m, x)) // 创建n行m列二维vector, 元素初始化为x

push_back(x)          // 在vector末尾添加元素
pop_back()            // 删除vector最后一个元素

insert(pos, x)        // 在pos位置插入元素
erase(pos)            // 删除pos位置元素
erase(begin, end)     // 删除区间[begin,end)内元素
v1.erase(v1.begin()); // 删除第一个元素
clear()               // 清空vector中的所有元素
resize(n)             // 改变vector大小, 不足补0, 多余删除
resize(n, x)          // 改变vector大小, 不足补x

a[i]                  // 下标访问元素
at(i)                 // 访问第i个元素, 越界抛异常
front()               // 返回第一个元素
back()                // 返回最后一个元素

size()                // 返回vector元素个数
empty()               // 判断vector是否为空, 空返回true
capacity()            // 返回vector当前容量
max_size()            // 返回最大可存储元素数量

begin()               // 返回头部迭代器
end()                 // 返回尾部后一个位置的迭代器
rbegin()              // 返回反向头迭代器
rend()                // 返回反向尾迭代器

sort(a.begin(), a.end()) // 升序排序 (需要algorithm)
sort(a.begin(), a.end(), greater<int>()) // 降序排序 (需要algorithm)
reverse(a.begin(), a.end()) // 反转vector元素顺序 (需要algorithm)
find(a.begin(), a.end(), x) // 查找元素x, 返回迭代器 (需要algorithm)
count(a.begin(), a.end(), x) // 统计元素x出现次数 (需要algorithm)
unique(a.begin(), a.end()) // 去除相邻重复元素, 通常配合erase使用

a.erase(unique(a.begin(), a.end()), a.end()) // vector去重 (需要先排序)

// ===== 遍历vector =====
for(int i = 0; i < v4.size(); i++) cout << v4[i] << " "; // 范围for遍历
for(int x : v4) cout << x << " "; // 增强for循环
for(auto it = v4.begin(); it != v4.end(); it++) cout << *it << " "; // 迭代器遍历

// ===== 赋值 =====
vector<int> a = {1, 2, 3}, b;
b = a; // 整体复制
b.assign(5, 10); // 将b重新赋值为5个10
b.assign(a.begin(), a.end()); // 使用迭代器范围重新赋值, b = {1, 2, 3}
```

cmath 数学公式

```
sqrt(x)          // 求平方根, 例如 sqrt(25)=5
cbrt(x)          // 求立方根
pow(x, y)        // 求  $x^y$ , 例如 pow(2,3)=8, 不适合用于大数取整, 需要自己写快速幂。不推荐用于取模 (会误差)

abs(x)           // 求整数绝对值, 例如 abs(-5)=5
fabs(x)          // 求浮点数绝对值, 例如 fabs(-3.14)=3.14

ceil(x)          // 向上取整, 例如 ceil(3.2)=4
floor(x)         // 向下取整, 例如 floor(3.8)=3
round(x)         // 四舍五入取整, 例如 round(3.5)=4
trunc(x)         // 去掉小数部分, 例如 trunc(3.8)=3
fmod(x, y)       // 浮点数取余, 例如 fmod(5.5,2)=1.5

max(x, y)        // 求两个数最大值 (也可用algorithm中的max)
min(x, y)        // 求两个数最小值 (也可用algorithm中的min)

// 三角函数, 参数必须是弧度
rad = angle * M_PI / 180.0;
sin(x)           // 正弦 (弧度)
cos(x)           // 余弦 (弧度)
tan(x)           // 正切 (弧度)
asin(x)          // 求反正弦值, 返回弧度
acos(x)          // 求反余弦值, 返回弧度
atan(x)          // 求反正切值, 返回弧度
atan2(y, x)      // 根据坐标(y, x)求角度, 返回弧度

log(x)           // 求自然对数 ln(x)
log10(x)         // 求以10为底的对数
exp(x)           // 求 e 的 x 次方

hypot(x, y)      // 求直角三角形斜边长度 sqrt(x*x+y*y)
```

set / multiset 升序

set: 自动去重

multiset: 不去重

底层是 红黑树, 插入、删除、查找复杂度均为 $O(\log n)$ 。

```
#include<set>

set<int> s;          // 定义一个空set (自动去重并升序排列)
set<int> s = {1,2,3}; // 初始化set

s.insert(x);         // 插入元素x (自动去重)
s.erase(x);         // 删除元素x
s.clear();           // 清空set中的所有元素
s.size();            // 返回set中元素个数
s.empty();           // 判断set是否为空, 空返回true

s.count(x);          // 判断元素x是否存在, 存在返回1, 否则返回0
s.find(x);           // 查找元素x, 返回迭代器, 找不到返回end()

s.begin();           // 最小值
s.rbegin();          // 最大值

s.lower_bound(x);     // 返回第一个大于等于x的元素的迭代器
s.upper_bound(x);     // 返回第一个大于x的元素的迭代器

s.swap(s2);          // 交换两个set的内容

for(auto x:s) cout << x << " ";
```

unordered_set 自动去重，不排序

底层 哈希表，查找顺序 $O(1)$ 。

除去与顺序相关的函数，其他函数与set基本一致。

```
#include <unordered_set>

unordered_set<int> s;           // 定义一个空unordered_set (自动去重, 无序)
unordered_set<int> s = {1,2,3}; // 初始化unordered_set

s.insert(x);                   // 插入元素x (已存在则不插入)
s.erase(x);                   // 删除元素x
s.clear();                    // 清空所有元素
s.size();                     // 返回元素个数
s.empty();                    // 判断是否为空, 空返回true

s.count(x);                   // 判断元素x是否存在, 存在返回1, 否则返回0
s.find(x);                   // 查找元素x, 返回迭代器, 找不到返回end()

s.begin();                   // 返回指向第一个元素的迭代器
s.end();                     // 返回末尾位置迭代器

s.swap(s2);                  // 交换两个unordered_set的内容

for(auto x:s) cout << x << " ";
```

map / multimap 键值对，key唯一，key升序

map：自动去重

multimap：不去重

底层 红黑树，按照key升序排列，查找、插入、删除复杂度 $O(\log n)$ 。

```
#include<map>

map<int,int> mp;           // 定义map, key和value类型均为int
map<int,string> mp;       // 定义key为int, value为string的map

mp[key]=value;            // 插入键值对, 如果key存在则修改value
mp.insert({key,value});   // 插入键值对
mp.erase(key);           // 删除key对应的键值对
mp.clear();               // 清空map中的所有元素
mp.size();               // 返回键值对数量
mp.empty();              // 判断map是否为空

mp.count(key);            // 判断key是否存在, 存在返回1, 否则返回0
mp.find(key);            // 查找key, 返回迭代器, 找不到返回end()

mp.begin();              // 返回指向第一个键值对的迭代器, 最小
mp.end();                // 返回末尾位置迭代器
mp.rbegin();             // 返回指向最大key的反向迭代器, 最大
mp.rend();               // 返回反向结束迭代器

mp.lower_bound(key);      // 返回第一个key ≥ key的迭代器
mp.upper_bound(key);      // 返回第一个key > key的迭代器

mp.swap(mp2);            // 交换两个map

// 增强for循环 + auto
for(auto x:mp){
    // 遍历map, x.first是key, x.second是value
    cout << x.first << " " << x.second;
}

// 迭代器遍历
map<int,string>::iterator it;
for(it = mp.begin(); it != mp.end(); it++){
    cout << it->first << " ";
    cout << it->second << endl;
}
```

unordered_map 哈希表 (去重, 不排序)

函数基本一致, 排序相关函数不可使用, 比如 `lower_bound()`

	map	unordered_map
底层	红黑树	哈希表
是否排序	✓ key自动排序	✗ 无序
查找	$O(\log n)$	平均 $O(1)$
是否唯一key	✓	✓
lower_bound	✓	✗

方法选择

- 需要按 key 排序 → map
- 只需要快速查找/计数 → unordered_map (更快)
- 需要保存插入顺序 → unordered_map + vector 或其他结构

pair

头文件

```
#include <utility>
```

相当于只有两个属性的结构体, `pair<int,int> p` 相当于如下结构体

```
struct Pair{
    int first;
    int second;
};
```

默认排序方式：升序排序

```
vector<pair<int,int>> v;

v.push_back({2,100});
v.push_back({1,200});

sort(v.begin(),v.end());
```

自定义排序：按照 second 降序排序

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <utility>
using namespace std;

bool cmp(pair<int,int> a, pair<int,int> b){
    return a.second > b.second;    // second大的排前面
}

int main(){
    vector<pair<int,int>> v;
    v.push_back({1,100});
    v.push_back({2,50});
    v.push_back({3,80});

    sort(v.begin(), v.end(), cmp);
    for(auto x:v)
        cout << x.first << " " << x.second << endl;
    return 0;
}
```

queue

```
#include <queue>
queue<int> q;           // 定义队列

q.push(x);              // 入队：从队尾加入元素x
q.pop();                // 出队：删除队头元素
q.front();              // 获取队头元素
q.back();               // 获取队尾元素
q.empty();              // 判断队列是否为空
q.size();               // 获取队列大小

// 遍历 or 清空队列
while (!q.empty()) {
    cout << q.front() << " "; // 获取队头元素
    q.pop();                 // 删除队头元素
}
```

stack

```
#include <stack>
stack<int> st;

st.push(10);    // 入栈
st.pop();       // 出栈
st.top();       // 栈顶
st.empty();     // 是否为空
st.size();      // 大小
```

也可以使用vector模拟栈

```
#include<iostream>
vector<int> st;

st.push_back(10); // 入栈
st.pop_back();    // 出栈
st.back();        // 栈顶
st.empty();
st.size();
```

priority_queue

大顶堆

```
#include <queue>
using namespace std;

priority_queue<int> q;           // 大顶堆，堆顶是最大值

q.push(x);                      // 插入元素 x
q.pop();                        // 删除堆顶元素
q.top();                        // 获取堆顶元素
q.empty();                      // 判断是否为空，空返回 true
q.size();                       // 获取元素个数
```

小顶堆

```
priority_queue<元素类型, 底层容器, 比较规则> q;
```

表示定义一个 `q`，里面存 `int`，底层用 `vector` 存储，并且按照 `greater<int>` 的规则维护堆。

```
#include <queue>
using namespace std;

priority_queue<int, vector<int>, greater<int>> q;    // 小顶堆，堆顶是最小值

q.push(x);                      // 插入元素 x
q.pop();                        // 删除堆顶元素
q.top();                        // 获取堆顶元素
q.empty();                      // 判断是否为空，空返回 true
q.size();                       // 获取元素个数
```

注意事项

注意事项

1. 标准C++不允许用变量定义数组长度，动态数组vector可以
2. 函数必须定义在main()函数前面，如果定义在main()后面需要在main()前面声明该函数
3. 工具 → 编译器选项 → 设置 `-std=c++11`



4. C++ 的函数返回类型不能直接写成数组类型

5. 数据类型最大值，对于有符号整数：

- `int` : 2.1×10^9 ($2^{31} - 1$)
- `long` : 9.2×10^{18} ($2^{63} - 1$)